

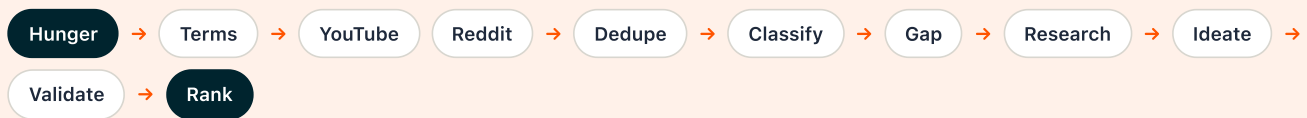


PIPELINE TRACE · DEMAND → ENRICHED FEED STORIES

RayDar *Engine*

How a batch travels from the **Hunger** seed, out through the YouTube and Reddit APIs, and back as ranked, evidence-grounded content ideas — every step, the code, and what it means.

The spine. Every ✦ AI step runs through the gated `runPipeline(id,...)` layer — provider · model · prompt from the swappable Admin registry, key from the encrypted Vault (BYOK), one log row per call. Every 🌐 fetch pulls its key from `getIntegrationKey(id)` and its query params from an editable business rule. Raw text never reaches a model outside the gate.



Nine stages, in pipeline order. Each shows **what the code is** (from `server/whisperer.js`) and **what it means**.

0

STAGE 0 Hunger — the demand seed ✦ HUNGER-GENERATE

`/api/wh/batch` · `/api/wh/hunger/:cohortId`

A batch fuses up to three routes — Trend-Spotting chips, pasted SEO text, and/or a TalentMind cohort — into a **Hunt Outcome**: who the cohort is, what they care about, and the **demand topics** to sweep.

— WHAT THE CODE IS

```
const routes = { trend: !!(trend_topics||[]).length, seo: !!seo, talentmind: !!talentmind_cohort_id };
hunger = { who: `Batch seeded from ...`, demand_topics: topics, cares_about: topics.slice(0,4), routes };

// cohort route → the Hunt Outcome is written by the gated pipeline over ClientMind chips
const { j, model } = await ai("hunger-generate", "Write the Hunt Outcome.",
  `Cohort: ${co.name}... Top chips: ${JSON.stringify(top)}`);
```

— WHAT IT MEANS

Before we touch any external API, the machine holds a structured picture of **demand** — topics plus the underlying questions the cohort is actually asking.

STAGE 1 Topic → search-term structuring

FAN-OUT

```
collectFeed(topics) · /api/wh/feedstories/:batchId
```

Each demand topic carries its own `terms[]`. `collectFeed` expands **topic → terms → individual queries** — those terms are the literal strings fired at every source, in parallel.

— WHAT THE CODE IS

```
for (const t of topics.slice(0,6)) {
  const terms = (t.terms?.length) ? t.terms : [t.name];
  for (const term of terms.slice(0,4)) {
    const batches = await Promise.allSettled([
      fetchYouTube(term), fetchReddit(term), fetchNews(term), fetchSerpNews(term)
    ]);
    ...out.push({ ...it, topic: name, term }); // tag every item with its concept
  }
}
```

— WHAT IT MEANS

One topic ("Salary negotiation") fans into ~4 concrete queries, each hitting 4 sources at once — `allSettled` means one source failing never sinks the sweep. Every item is stamped with its parent topic so gap-analysis can regroup later.

STAGE 2 YouTube — three chained calls

SEARCH · STATS · COMMENTS

```
fetchYouTube(topic) – key + params from the youtube business rule
```

— WHAT THE CODE IS

```
// 2a – search fresh, high-view videos
fetch(`.../youtube/v3/search?type=video&order=viewCount@ionCode=IN
      &publishedAfter=${...}&q=${term}&key=${key}`)
// 2b – video stats (views + counts + date) → velocity
fetch(`.../youtube/v3/videos?part=statistics,snippet&id=${ids}&key=${key}`)
  → stats[id] = { views, comments, publishedAt }
// 2c – top-N comment threads (the real demand signal), in parallel
fetch(`.../youtube/v3/commentThreads?videoId=${v.id}&order=relevance`)
  → { source:"youtube", title, url, meta:{ views, ageDays, comments:[...] } }
```

— WHAT IT MEANS

We don't just count videos — we capture **views ÷ age** (velocity) and the **comment text** (what people are actually confused, angry or excited about). Comments carry the feeling; stats carry the momentum.

STAGE 3 Reddit — OAuth → search → walk comment trees

 PAIR-AUTH

fetchReddit(topic) – client_id:client_secret pair from the Vault

— WHAT THE CODE IS

```
// 3a – app-only OAuth token
fetch(".../api/v1/access_token", { headers:{ authorization:"Basic "+base64(`${cid}:${secret}`) },
  body:"grant_type=client_credentials" })
// 3b – search posts (subreddits / timeframe from the rule)
fetch(`...oauth.reddit.com/search?q=${term}&sort=relevance&t=month`, { headers:{ Bearer... } })
// 3c – recursively walk the comment tree on the top-N posts
const walk = (node) => { for (const ch of node?.data?.children||[]) {
  if (ch.kind==="t1" && ch.data?.body) { arr.push(ch.data.body); walk(ch.data.replies); } } };
→ { source:"reddit", title, url, meta:{ score, ageDays, comments:[...] } }
```

— WHAT IT MEANS

Reddit is treated as a **leading indicator** (a guardrail caveat, not a census). The recursive **walk** mines nested replies — where the honest, high-signal complaints live — and normalises them into the **same item shape** as YouTube, so everything downstream is source-agnostic.

STAGE 4 Classify — channel each item through its rule prompt

 RAYDAR-CLASSIFY

classifyFeed(items) – per-source prompt + model + gate · also dedupe by URL & persist

— WHAT THE CODE IS

```
// first: dedupe by url + persist each item into wh_feed_item
const uniq = out.filter(x => x.url && !seen.has(x.url) && seen.add(x.url));

runPipeline("raydar-classify", {
  system: `${rule.prompt}\nReturn STRICT JSON {"items":[{"i","topic","franchise",
    "registers":{"FOMO,Anxiety,Optimism,Ambition},"question"}]}`,
  user: payload, provider: prov, model: mdl });
→ arr[t.i].tags = t; // tag each item
```

— WHAT IT MEANS

Raw feed → tagged feed. Each item gets a demand topic, a 1Up franchise, a 4-register emotional distribution, and the **underlying question** it represents — all decided by the swappable per-source prompt, never hardcoded.

5

STAGE 5 Gap Analysis — pure math, no model

DETERMINISTIC

topicSignals(items, gapMapVal)

— WHAT THE CODE IS

```
velocity = log10(max(views/ageDays)+1)/5           // momentum, ~100k/day → 1
demand   = # question-like comments + titles       // /how|why|which|worth it|vs/
supply   = item count × freshness                  // halved if top item > 365 days stale
gap       = clamp(demandN / supplyQ)                // hunger ÷ what's already answered
gapType  = stale | emerging | unanswered | thin
```

— WHAT IT MEANS

The money step. **Gap = demand ÷ supply**. A topic with 20 unanswered questions and only 2 stale videos scores a high gap (big opportunity); a saturated topic scores low. Deterministic and auditable — it falls back to the configured `gap_map` when live metrics are missing.

6

STAGE 6 Research — grounding sources in parallel

TAVILY · SERPER · PERPLEXITY · EXA · BRAVE

researchTopic(topic) — every enabled source, each reading its own rule

— WHAT THE CODE IS

```
if (enabled("tavily")) runs.push(/* POST api.tavily.com/search → answer + refs[] */);
if (enabled("perplexity")) runs.push(/* sonar chat → cited answer */);
...
for (const s of await Promise.allSettled(runs))
  { out.refs.push(...s.value.refs); out.sources.push(s.value.source); } // each ref tagged
```

— WHAT IT MEANS

Separate from the **demand** feed, this pulls **authoritative** material to ground and fact-check the idea, with citable URLs — every reference attributed to its origin.

7

STAGE 7 Ideate — three angles per topic, grounded

FEEDSTORY-GENERATE

core · contrarian · insider-data — with Talent500 guardrails prepended

— WHAT THE CODE IS

```
ANGLES = ["core", "contrarian", "insider-data"]; // a===1 → contradiction idea
ai("feedstory-generate",
  `${guardPrompt}\nCreate ONE content idea (heading + topic-guide brief, NOT finished
    copy) for '${t.franchise}'. Angle: ${ANGLES[a]}. Ground it in the real feed + research.` ,
  `Demand topic: ${t.name} (question: ${t.question})... Real feed: ${items} Research: ${...}`);
if (!j || !j.heading) continue; // LLM-only, no dummy fallback
```

— WHAT IT MEANS

Output is a **brief for a writer** (heading + topic guide), never finished copy — one straight take, one contrarian, one data-led — each anchored to specific feed evidence.

8

STAGE 8 Contradiction & Evidence — validate the idea

RAYDAR-CONTRADICTION

validateIdea(story, research)

— WHAT THE CODE IS

```
runPipeline("raydar-contradiction", {
  system: `Return {"evidence":[{"claim,source}], "contradictions":[{"claim,conflict}]}`.
  Only cite what research supports; never assert independently.` });
// a conflict → flag the story and record the belief it pushes against
if (val.contradictions.length) { s.contradiction = true; s.contradiction_of = ...conflict; }
```

— WHAT IT MEANS

The "you propose, never assert" guardrail in code — claims get citations; a wrong-but-popular belief becomes a **contrarian angle with evidence**, not an unbacked hot take.

9

STAGE 9 Rank — composite score, then persist

Σ SIGNALS → WH_FEED_STORY

weighted sum from the editable scoring rule + the human feedback loop

— WHAT THE CODE IS

```
score = W.gap*gap + W.velocity*velocity + W.strategic*strategic + W.historical*franchiseHist;
// default weights: gap .35 · velocity .25 · strategic .20 · historical .20
// historical = past "Used" acceptance per franchise → feedback feeds back into rank
await wq(`insert into wh_feed_story(... score, score_breakdown ...)`); // explainable, stored
```

— WHAT IT MEANS

Every card is ranked by **opportunity × momentum × strategy × what's worked before**, and the full breakdown (demand, supply, live-flag, sources) is stored — so the ranking is completely explainable in the report, never a black box.

Σ

The whole chain, one line

Hunger (demand topics) → **terms** → parallel **YouTube** (search·stats·comments) + **Reddit**

(OAuth·search·comment·trees) + News/Serp → **dedupe/persist** → **classify**

(topic·franchise·register·question) → **gap = demand ÷ supply** + velocity → **research grounding** → **ideate** (3 guardrailed angles) → **contradiction/evidence** → **composite rank** → stored, explainable Feed Stories.



Bespoke AI by The Kettle Black

RayDar — Talent Trend Radar for Q&ANSR

RayDar Engine · pipeline trace

Source: server/whisperer.js

Generated 24 Jul 2026